

UNIVERSIDADE DA CORUÑA

Práctica 2: Comunicación y comportamiento social

Sistemas Multiagente
Grado en Inteligencia Artificial

Guillermo Blanco Núñez
Pablo Díaz Blanco

Grupo Martes
3 de mayo de 2026

Contents

1	Mundo Virtual	2
1.1	Estructura y distribución del escenario	2
1.2	Agentes del escenario	4
1.3	Interacción con visión y sonido	5
1.3.1	Percepción auditiva	5
1.3.2	Percepción visual	5
1.3.3	Sensores específicos	6
2	Arquitectura individual	6
2.1	Guardias: capa reactiva (subsumption)	7
2.2	Guardias: capa deliberativa (BDI)	8
2.3	Ladrón: arquitectura PECA	9
3	Comunicación entre agentes	10
3.1	Estructura de los mensajes FIPA-ACL	10
3.2	Infraestructura de comunicación	11
3.3	Protocolos implementados	11
4	Formato del contenido de los mensajes	13
5	Estrategia Multiagente	14
5.1	Mecanismos de coordinación y roles dinámicos	14
5.2	Respuesta en crisis y búsqueda colaborativa	15
5.3	Propiedades del sistema	15

Mundo Virtual

Hemos optado por un enfoque continuista respecto a la P1 en cuanto al mundo virtual. Sin embargo, se ha optado por una disposición más compacta: se ha reducido el espacio dentro de las vallas y se han añadido más agentes, lo que hace más relevante la coordinación multiagente. Fuera del recinto de la villa, el cambio más destacado es la incorporación de una segunda hoguera destino que define la condición de victoria.

Estructura y distribución del escenario

- El mundo virtual representa una aldea medieval amurallado que el jugador (ladrón) debe infiltrar. El escenario incluye la muralla de madera, tiendas de campaña, casas, caminos con pasarelas de madera y una zona de cultivos.
- El pueblo tiene una única entrada y salida: una puerta corredera lateral (**SlidingGate**) situada en la muralla. Cuando el ladrón entra en su zona de activación, la puerta se abre de forma permanente y ya no vuelve a cerrarse. Que sea la única entrada y salida la convierte en el objetivo estratégico principal de los guardias cuando se activa la alarma, ya que aunque el ladrón robe el fuego no puede ganar la partida hasta que no sale de la aldea y enciende su propia hoguera.
- Las casas disponen de puertas inteligentes (**Intelligent_Door_Houses**) que se abren automáticamente por proximidad mediante un trigger. Al abrirse, la puerta desliza lateralmente y desactiva su **NavMeshObstacle** para permitir el paso de agentes; al cerrarse lo reactiva, restaurando el obstáculo de navegación.
- Hay dos hogueras con roles diferenciados: la **hoguera origen**, en el centro del pueblo, es donde el ladrón debe encender su antorcha robada y robar el fuego. La **hoguera destino** es la meta final; depositar el fuego robado en ella activa la condición de victoria. Es también en la ubicación de esta hoguera destino donde aparece el jugador (ladrón) al iniciar el juego.

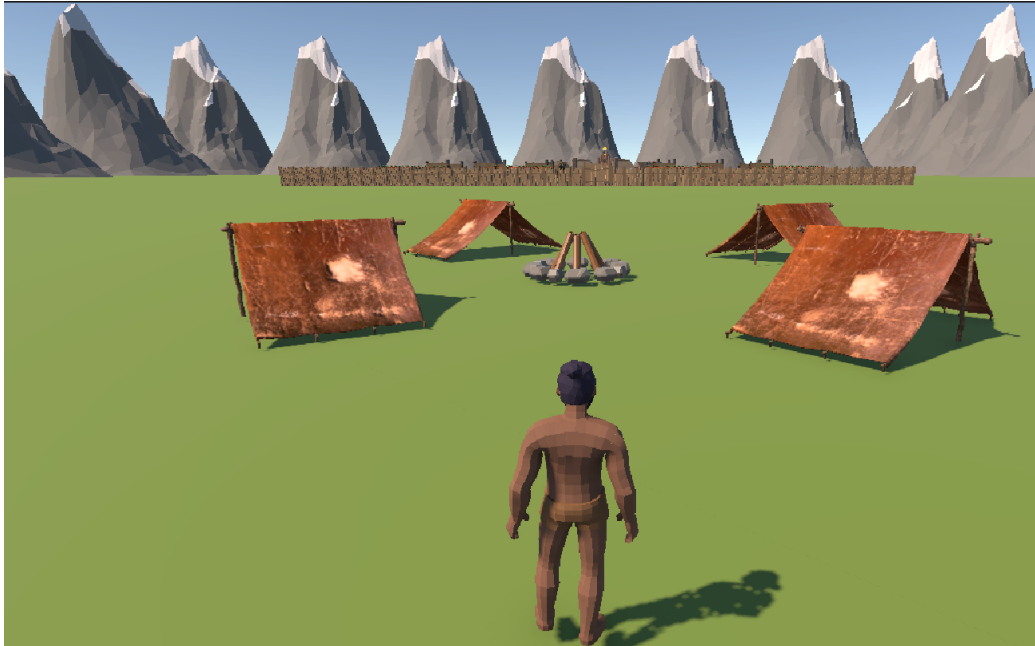


Figure 1: Zona de la hoguera destino (apagada) al inicio de la partida, con el jugador delante y el poblado donde debe entrar de fondo.

- El mapa está dividido en **sectores de búsqueda**, representados como **GameObjects** con el tag **Sector**. Son cargados al inicio por **SectorMap** y usados por la capa deliberativa para planificar búsquedas coordinadas cuando el ladrón no está a la vista.



Figure 2: Vista cenital del poblado. Con la entrada a la izquierda y la hoguera de la cual el jugador debe robar el fuego en la plaza de la derecha.

Agentes del escenario

Contamos con nueve agentes en el mundo virtual, además del jugador (ladrón). Además de mantener los siete que teníamos en la práctica 1, añadimos dos más. Tenemos tres tipos de agentes:

- **Aldeanos (6)** - tienen posiciones iniciales distribuidas por el pueblo, cada uno con una ruta de patrulla propia formada por puntos de control que recorren de forma cíclica. Hay dos en la entrada, en la zona de las tiendas de campaña, dos en la zona de las casas, y otros dos en la zona de la plaza donde la hoguera. Estos dos últimos tienen también sensor de oído y comportamientos especiales relacionados con la hoguera.
- **Campeños (2)** - se encuentran en la zona de los cultivos, uno a cada lado, tienen comportamientos similares a los aldeanos pero estos cuentan ambos con sensor de oído.
- **Lobo (1)** - Se encuentra patrullando muy cerca de la puerta de entrada al poblado, por fuera. Es el único agente sin la posibilidad de abrir puertas de casas ni del poblado. No participa en la comunicación ni en

la estrategia multiagente, pero representan un peligro adicional para el jugador incluso antes de entrar en el poblado y también cuando sale del poblado con el fuego y quiere ir a la hoguera destino para ganar la partida.

La condición de derrota se gestiona mediante **DetectorCaptura**: cuando uno de los agentes entra en contacto con el ladrón, se dispara el evento de captura y la escena se reinicia tras 1,5 s.

Interacción con visión y sonido

Hemos cambiado significativamente los sensores de los agentes con respecto a la práctica 1 para que sean lo más realistas posibles.

1.3.1 Percepción auditiva

El **sonido** es simulado mediante un radio de detección numérico asociado al movimiento del ladrón: 0 m cuando está quieto, 10 m cuando corre y 15 m cuando salta. Este valor es leído cada frame directamente por el sensor **GuardHearing** de cada guardia.

1.3.2 Percepción visual

La **visión** de los guardias es un cono de 45° de apertura y 15 m de alcance. El sensor **GuardVision** realiza primero un **OverlapSphere** para obtener candidatos en rango y luego un raycast hacia cada candidato para verificar que no hay obstáculos físicos (muros, casas) en la línea de visión. También detecta si el ladrón lleva fuego comprobando el tag "**LadronConFuego**".

El **sensor de hoguera** (**SensorHogueraIndividual**) tiene un campo de visión más amplio: 120° de apertura y 12 m de radio. La primera vez que el guardia la detecta, memoriza su posición de forma permanente (**haVistoHogueraAlgunaVez**). En frames sucesivos comprueba esa posición memorizada con un raycast, permitiendo detectar su desaparición aunque la hoguera no esté directamente en el campo de visión. El raycast usa **RaycastAll** para atravesar objetos decorativos sin colisión real.

Los obstáculos decorativos (casas, cultivos, objetos, etc.) se configuran en capas específicas (**capaObstaculos**) de modo que bloqueen la visión solo cuando representan una ocultación real, y sean transparentes para los raycasts en los demás casos.

1.3.3 Sensores específicos

El sensor de puertas (`SensorPercepcionObjetos`) realiza un `OverlapSphere` buscando objetos con el tag "Doors_houses", lo que permite al guardia detectar y acercarse a comprobar puertas cercanas durante la búsqueda.

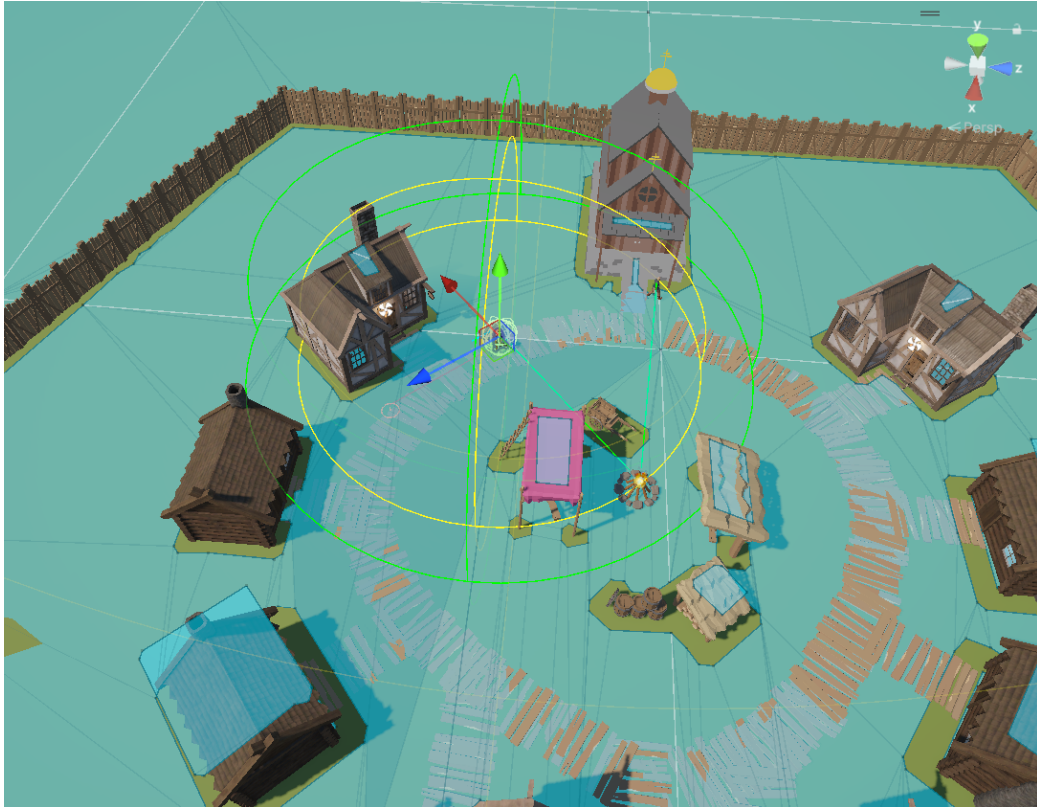


Figure 3: Visualización en tiempo de ejecución de los sensores de un guardia en la ventana *Scene* de Unity con Gizmos activados: cono de visión de 45° y 15 m (`GuardVision`, verde), radio de audición variable (`GuardHearing`, amarillo) y línea de detección de hoguera (`SensorHogueraIndividual`, verde).

Arquitectura individual

La arquitectura individual de los guardias ha experimentado una evolución considerable desde la práctica 1. En P1 se empleaba una arquitectura reactiva basada en una máquina de estados con cinco comportamientos (patrulla, persecución, búsqueda, investigación de entorno y bloqueo de salida) y con la comunicación entre agentes expresamente prohibida. Para P2 esa estructura

se ha formalizado y ampliado: la capa reactiva se convierte en una arquitectura de subsumption con siete comportamientos, la lógica temporal y de gestión de estado se extrae en módulos independientes, y se añade una capa deliberativa BDI que, por primera vez, permite que los guardias se coordinen mediante mensajes FIPA-ACL. El ladrón, en cambio, mantiene en esencia la misma arquitectura de la P1, con ajustes en la mecánica de inventario para la secuencia de victoria en dos hogueras.

Guardias: capa reactiva (subsumption)

Se adopta una **arquitectura híbrida** que combina una capa reactiva (subsumption) con una capa deliberativa (BDI). La capa reactiva gestiona las respuestas inmediatas al entorno; la deliberativa, la coordinación entre agentes.

La **capa reactiva** sigue la arquitectura de subsumption de Brooks: un conjunto de comportamientos ordenados por prioridad donde el de mayor prioridad cuyo `CanActivate()` retorna `true` suprime a todos los inferiores y ejecuta su `Action()` en ese frame. No existe máquina de estados explícita: la prioridad reemplaza las transiciones.

Los **siete comportamientos**, de mayor a menor prioridad:

1. **Persecución** (priority = 0): activo cuando el guardia ve al ladrón o lo perdió mientras llevaba fuego. Persigue la última posición conocida (10 m/s con fuego, 8 m/s sin él).
2. **BloquearSalida** (priority = 1): activo cuando la alarma de que falta la hoguera está encendida. El guardia corre a la puerta del pueblo (10 m/s) y la bloquea vigilando el interior.
3. **InvestigarSonido** (priority = 2): activo al detectar ruido y sin cooldown. El guardia va a la posición estimada del ruido (imprecisión ± 5 m).
4. **ComprobarHoguera** (priority = 3): activo cuando la búsqueda se agota y no hay alarma. El guardia camina físicamente hasta la hoguera para verificar que sigue ahí.
5. **InvestigarEntorno** (priority = 4): activo si hay una puerta detectada y el cronómetro de búsqueda está en curso. El guardia se acerca a la puerta para comprobarla.
6. **Busqueda** (priority = 5): activo en alerta cuando no ve al ladrón pero tiene su última posición. Busca en radio ~ 5 m; cada 3 ciclos amplía el radio a 25 m desde la hoguera.

7. Patrulla (*priority* = 6): comportamiento por defecto. Recorre sus puntos de control cíclicamente a 2 m/s.

El **SubsumptionController** actúa como orquestador central: cada frame sincroniza percepciones, actualiza el estado de alerta, gestiona cronómetros y toma la decisión final de comportamiento. Toda la información del agente se centraliza en el **AgentBlackboard**, que almacena las creencias del frame actual (posición del ladrón, alertas, ruidos, puertas, etc.) y es accesible por todos los módulos. El **AlertCycleManager** controla las transiciones de alerta, incluyendo la persecución hasta la última posición conocida si el ladrón porta fuego. El **AgentTimerManager** mantiene cronómetros independientes para búsqueda (10 s), investigación de sonidos (5 s) y puertas (8 s). Por último, el **FireAlarmMonitor** detecta la desaparición de la hoguera tras 10 frames sin detectarla y activa la alarma correspondiente.

Guardias: capa deliberativa (BDI)

La **capa deliberativa** (**DeliberativeLayer**) se ejecuta cada frame, antes de la decisión reactiva, y está compuesta por tres módulos especializados:

- **ModuleCrisis**: detecta mensajes de alarma de hoguera (**INFORM "alarma_hoguera"**) y la difunde al resto de guardias mediante broadcast cuando se activa.
- **ModuleSocial**: implementa el Contract Net Protocol (CNP) para negociar roles de perseguidor y cubridor, gestionando los estados de la negociación y el intercambio de mensajes **CFP**, **PROPOSE** y **ACCEPT_PROPOSAL**.
- **ModuleTactical**: calcula distancias para las propuestas del CNP, inyecta los comandos resultantes en el **SubsumptionController** y ordena los sectores de búsqueda mediante **A***.

La **BeliefBase** de la capa deliberativa mantiene creencias de largo plazo independientes del blackboard reactivo: la posición del ladrón se olvida tras 10 s sin avistarlo, y se registran además si porta fuego, si la alarma está activa y el plan de sectores de búsqueda asignado. La integración con las comunicaciones es inmediata: el **AgentCommunicator** procesa los mensajes entrantes cada frame y los resultados de la negociación se inyectan directamente en el **SubsumptionController**, alterando el blackboard y, por tanto, el comportamiento reactivo en ese mismo frame.

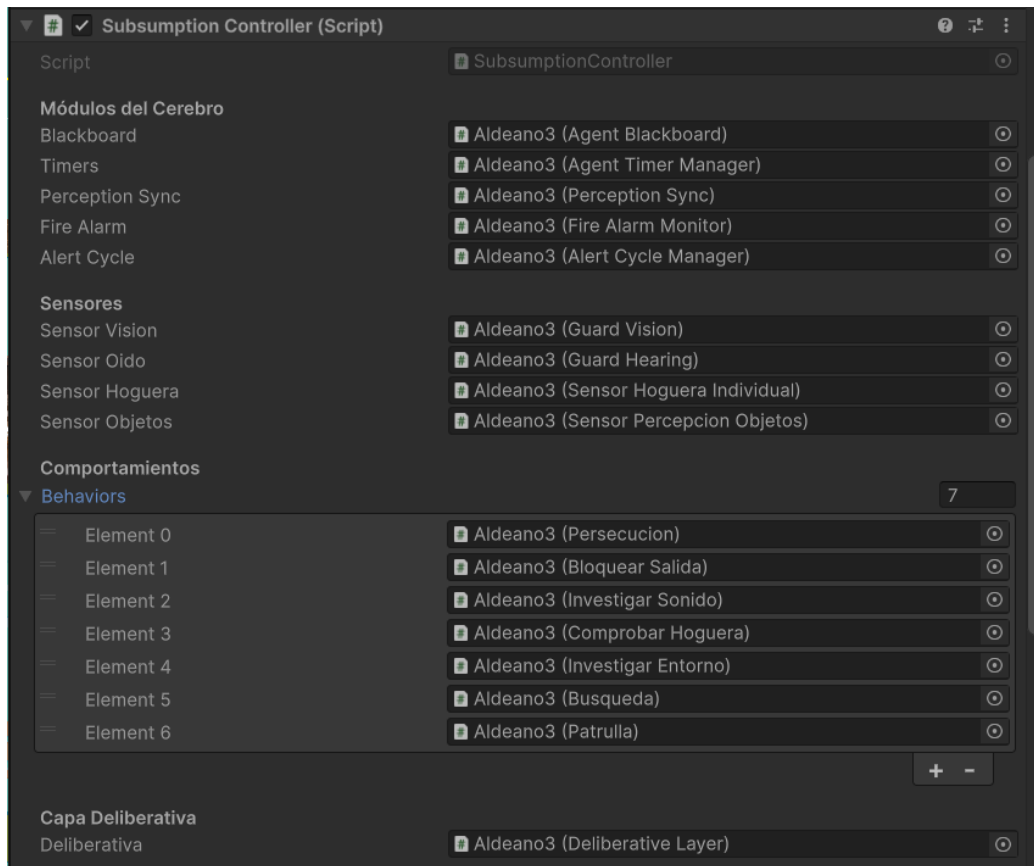


Figure 4: Inspector de Unity del `SubsumptionController`: componentes de la arquitectura híbrida y jerarquía de comportamientos.

Ladrón: arquitectura PECA

El ladrón sigue una arquitectura **PECA** (Percepción → Estado → Consideración → Acción), implementada en `MainCharacter_Brain.cs`.

Percepción Lectura del input del jugador cada frame: WASD para moverse, ratón para rotar la cámara, espacio para saltar, E para recoger o interactuar y F para depositar.

Estado Comprobación del estado físico (`isGrounded`) y del bloqueo por animación (`isActionLocked`), que impide interrumpir una acción en curso.

Consideración Cálculo del radio de ruido según el movimiento: 0m en reposo, 10m corriendo y 15m saltando. Este valor se almacena en `radioRuidoActual` y es leído directamente por `GuardHearing` sin mediación de mensajes.

Acción Movimiento físico mediante `CharacterController` con gravedad y salto simulados, rotación de cámara e interacciones de recogida y depósito a través de callbacks de animación.

Secuencia de victoria

La condición de victoria se desarrolla en tres pasos:

1. Recoger la antorcha (tecla E)
2. Encenderla en la hoguera origen (tecla E), lo que cambia el tag del `GameObject` a "LadronConFuego" y activa la detección de urgencia en los guardias
3. Depositar el fuego en la hoguera destino (tecla F), activando los efectos de victoria y pausando el juego.

Comunicación entre agentes

La comunicación entre agentes es la principal novedad de esta práctica respecto a la P1, donde los agentes operaban de forma completamente aislada. Se ha implementado un sistema de mensajería basado en el estándar FIPA-ACL con dos protocolos diferenciados: un protocolo de notificación simple mediante mensajes `INFORM` (para difusión de posición del ladrón y propagación de alarmas) y el Contract Net Protocol (CNP) para la negociación y asignación dinámica de roles entre guardias.

Estructura de los mensajes FIPA-ACL

- Los mensajes siguen el estándar **FIPA-ACL** y están implementados en la clase `FIPAMessage`, con los campos: `performativa` (tipo de acto comunicativo), `emisor` y `receptor` (nombres de los agentes como strings), `contenido` (payload en texto plano), `conversationId` (identifica el hilo de una negociación), `timestamp` (`Time.time` en el momento del envío) e `inReplyTo` (referencia al mensaje al que se responde).
- Se implementan **seis performativas** (enum `FIPAPerformativa`): `INFORM` (notificación de hechos), `CFP` (*Call For Proposals*), `PROPOSE` (oferta de un contratista), `ACCEPT_PROPOSAL` (aceptación y asignación de rol), `REFUSE` (rechazo de participación) y `FAILURE` (notificación de fallo).
- Los mensajes se almacenan en un **historial persistente** (`historialMensajes`) y pueden recuperarse agrupados por conversación mediante

`GetHistorialPorConversacion(conversationId)`, lo que permite reconstruir el hilo completo de una negociación.

Infraestructura de comunicación

- La infraestructura es **P2P sin bus central**: `AgentCommunicator` mantiene un registro global estático (`Dictionary<string, AgentCommunicator>`). Cada agente se registra automáticamente en `Start()` con su nombre y se desregistra en `OnDestroy()`. No existe componente intermediario centralizado.
- **Flujo de envío**: el emisor llama a `Enviar(mensaje, destinatarios)` para envío unicast o a `EnviarATodos(mensaje)` para broadcast. El sistema busca a cada destinatario en el registro y encola el mensaje directamente en su cola **pendientes**.
- **Flujo de recepción**: en cada frame, `AgentCommunicator` vacía **pendientes** hacia el **inbox** del frame. `DeliberativeLayer.Procesar()` lee el **inbox** al inicio de su ejecución y despacha cada mensaje al módulo correspondiente (`ModuleCrisis` o `ModuleSocial`) según la performativa y el contenido.
- El `conversationId` sigue el formato "agente-tipo-contador-tiempo", generado de forma única por `AgentCommunicator.GenerarConversationId()` al iniciar cada negociación.

Protocolos implementados

- **Protocolo 1 — INFORM periódico de posición**: cuando un guardia tiene al ladrón a la vista, el `SubsumptionController` envía periódicamente (cada 0,5s) un **INFORM** con la posición del ladrón a los demás guardias. Esto permite que los guardias que no lo ven directamente actualicen sus creencias y respondan con mayor rapidez.
- **Protocolo 2 — INFORM de alarma hoguera**: cuando `FireAlarmMonitor` confirma la ausencia de la hoguera, activa la alarma localmente y `ModuleCrisis` envía un **INFORM** broadcast con contenido "alarma_hoguera" a todos los demás guardias. Al recibirlo, `ModuleCrisis` de cada receptor inyecta la alarma en su propio `SubsumptionController`, propagando la crisis de forma reactiva e inmediata sin negociación.
- **Protocolo 3 — Contract Net Protocol (CNP)**: se activa cuando un guardia avista al ladrón (`AcabaDeVerAlladron == true`) y no hay ya una negociación en curso. El guardia que lo detecta actúa como **manager**:

1. El manager envía un CFP broadcast indicando la posición del ladrón y el objetivo a cubrir (hoguera o salida, según si el ladrón lleva fuego).
2. Cada guardia receptor (**contractor**) calcula su distancia al ladrón y al objetivo, y responde con PROPOSE. Si ya está en otra negociación, responde REFUSE.
3. Tras un tiempo de espera de 0,5 s, el manager evalúa las propuestas, designa el perseguidor (mínima distancia al ladrón) y el cubridor (mínima distancia al objetivo), y envía ACCEPT_PROPOSAL a cada uno con el rol asignado en el campo contenido.
4. Los guardias seleccionados ejecutan el rol recibido: **ModuleTactical** inyecta "perseguir:x,y,z" para el perseguidor o "cubrir_hoguera"/"cubrir_salida" para el cubridor en el **SubsumptionController**.

La modificación sobre el CNP estándar consiste en la asignación simultánea de dos roles diferenciados (perseguidor y cubridor) en un único ciclo de negociación. El CNP estándar asigna una tarea a un único contractor; aquí se seleccionan dos contractors óptimos de forma simultánea con una sola ronda de mensajes.

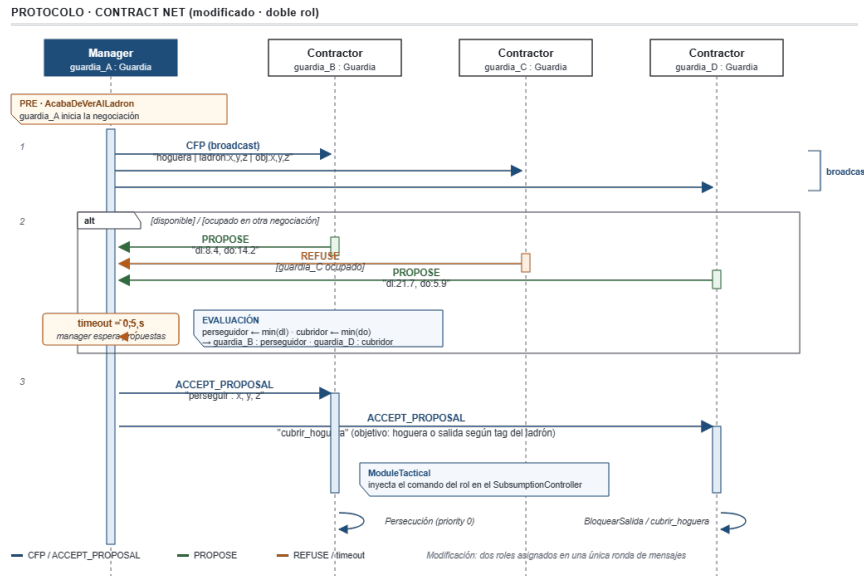


Figure 5: Diagrama AUML del Contract Net Protocol modificado: el manager asigna simultáneamente un rol de perseguidor y un rol de cubridor en una única ronda de negociación

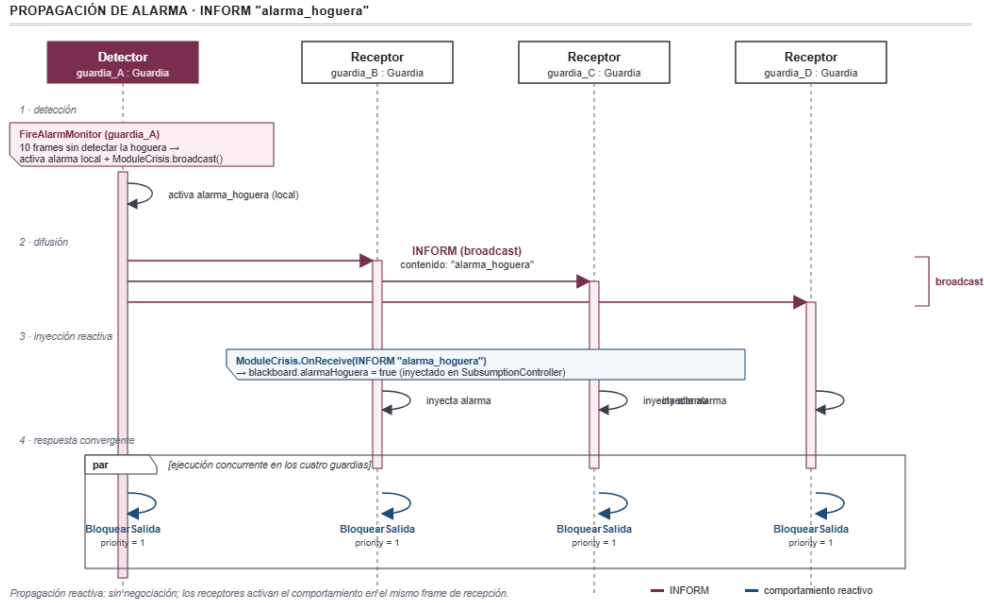


Figure 6: Flujo de comunicación para la propagación reactiva de la alarma de hoguera mediante mensajes INFORM broadcast

Formato del contenido de los mensajes

El contenido de los mensajes se representa como texto plano en español en el campo `string` de `FIPAMessage`, sin JSON, XML ni KQML, lo que reduce el coste de serialización y facilita la depuración durante el desarrollo.

Formatos por performativa

El formato del campo `contenido` varía según la performativa:

INFORM posición del ladrón "ladron:x,y,z" — clave fija más las tres coordenadas float separadas por comas.

INFORM alarma hoguera Literal fijo "alarma_hoguera", comprobado por igualdad de cadena sin parsing adicional.

CFP "tipo|ladron:x,y,z|obj:x,y,z" — tres campos separados por |. El primero es el tipo de misión ("hoguera" o "salida"), el segundo la posición del ladrón y el tercero la posición del objetivo a cubrir.

PROPOSE "dl:valor,do:valor" — dos pares clave-valor separados por

coma; `dl` es la distancia euclídea al ladrón y `do` la distancia al objetivo, ambos como floats.

ACCEPT_PROPOSAL (perseguidor) "perseguir:`x,y,z`" — acción fija más las coordenadas de la última posición conocida del ladrón.

ACCEPT_PROPOSAL (cubridor hoguera) Literal fijo "cubrir_hoguera".

ACCEPT_PROPOSAL (cubridor salida) Literal fijo "cubrir_salida".

Procesamiento y deserialización

El procesamiento emplea operaciones básicas de string: `string.Split('|')` para los campos del CFP, `string.Split(':')` para separar clave y valor, y `string.Split(',')` para las coordenadas individuales. Los literales fijos se comprueban con `string.Equals()` o el operador `==`. La conversión de coordenadas a `Vector3` está encapsulada en `ParsearPosicion()` de `ModuleTactical`, centralizando toda la lógica de deserialización en un único lugar.

Gestión del hilo de conversación

El campo `conversationId` se incluye en todos los mensajes de un mismo hilo (CFP, PROPOSE, ACCEPT_PROPOSAL), permitiendo recuperar el historial completo de una negociación. El campo `inReplyTo` almacena el `conversationId` del mensaje original al que se responde, estableciendo la cadena causal entre mensajes.

Estrategia Multiagente

La estrategia cooperativa gira en torno a un objetivo común: impedir que el ladrón escape con el fuego de la hoguera. Para lograrlo, los guardias combinan comportamiento reactivo coordinado (a través de mensajes `INFORM`) con negociación deliberativa de roles (mediante el CNP), de modo que la cooperación emerge de las interacciones locales entre agentes sin necesidad de un coordinador global.

Mecanismos de coordinación y roles dinámicos

La estrategia se articula en dos mecanismos complementarios: negociación de roles mediante CNP para coordinar la respuesta cuando el ladrón es avistado, y propagación reactiva de alarma mediante `INFORM broadcast` para la fase de crisis.

No existe jerarquía fija entre guardias: cualquier guardia puede actuar como manager del CNP al avistar al ladrón. Los roles de perseguidor y cubridor se asignan en cada ronda en función de las distancias en ese momento. El manager designa como **perseguidor** al guardia con menor distancia euclídea al ladrón y como **cubridor** al de menor distancia al objetivo a proteger; si ambos roles corresponden al mismo guardia, ese guardia cubre el objetivo y el resto persigue.

El objetivo de cobertura cambia dinámicamente según el estado del ladrón: si no lleva fuego, se cubre la hoguera para evitar que la encienda; si porta el tag "**LadronConFuego**", se cubre la puerta de salida para evitar que escape. Esta lógica se codifica directamente en el tipo del CFP.

Respuesta en crisis y búsqueda colaborativa

Cuando un guardia detecta que la hoguera ha desaparecido, activa la alarma localmente y la difunde mediante **INFORM broadcast**. Todos los guardias que reciben el mensaje activan **BloquearSalida** de máxima prioridad y convergen hacia la única salida del pueblo. La respuesta es puramente reactiva, no requiere nueva negociación, y se propaga en el mismo frame en que se recibe el mensaje.

Cuando ningún guardia avista al ladrón pero hay alerta activa, **ModuleTactical** genera un plan de búsqueda ordenando los sectores del mapa mediante un algoritmo A^* que minimiza la distancia total recorrida (solución aproximada al TSP). Los sectores se almacenan en la **BeliefBase** y se inyectan en el comportamiento **Busqueda**, guiando la exploración de forma sistemática en lugar de aleatoria.

Propiedades del sistema

El sistema opera sin coordinador central: cualquier guardia puede iniciar un CNP, y el timeout de 0,5s garantiza que la negociación se resuelve aunque algún contractor no responda. El flag **cnpIniciado** evita negociaciones solapadas en el mismo guardia, y el mensaje **REFUSE** permite que guardias ya ocupados rechacen participar sin bloquear el sistema. Para evitar bucles de persecución indefinida, el **AlertCycleManager** tras 3 ciclos de búsqueda interrumpidos fuerza una comprobación física de la hoguera, garantizando que los guardias verifiquen su objetivo principal. Aunque el ladrón carece de estrategia multiagente, actúa como estímulo central del sistema: su visibilidad activa el CNP, su cambio de tag al llevar fuego modifica el objetivo de cobertura, y su ruido variable puede desviar guardias mediante **InvestigarSonido**, creando ventanas de oportunidad para el jugador.